

Student Eligibility & Debarment Checker!

Vityarthi Project Report

**CSE1021 – Introduction to Problem Solving and
Programming**

Submitted by: Samiksha Saini

Registration Number: 26BHI10123

Faculty: Prof. Adarsh Patel

Date: September 2026

GitHub Repository: <https://github.com/samiksha-pixel97/Student-Eligibility-Checker>

List Of Contents:

- **Student and Faculty Introduction**
- **Introduction**
- **Problem Statement**
- **Functional Requirements**
- **Non-Functional Requirements**
- **System Architecture**
- **Design Diagrams**
- **Design Decisions and Rationale**
- **Implementation Details**
- **Screenshots/Results**
- **Testing Approach**
- **Challenges Faced**
- **Learnings and Key Takeaways**
- **Future Enhancements**
- **References**

Introduction:

Every semester, colleges set minimum attendance and internal assessment requirements that a student must meet in order to sit for their end-semester examinations. In practice, checking this for an entire class means going through attendance registers and internal marks sheets one student at a time, applying the same set of rules again and again. It's repetitive, and it's the kind of task that's easy to get wrong when done by hand for a large batch.

This project, the Student Eligibility & Debarment Checker, was built to automate exactly that process using Python. It takes each student's attendance percentage and internal marks, applies the college's eligibility rules, and produces a clear status for every student — along with a class-wide summary. The project was built for the course CSE1021 – Introduction to Problem Solving and Programming, and it deliberately sticks to the concepts covered in that course: conditionals, loops, functions, lists, dictionaries, and basic sorting/searching algorithms, rather than reaching for tools outside the syllabus.

Problem Statement

Colleges require students to meet minimum attendance and internal assessment thresholds to be eligible for end-semester examinations. Manually checking every student against multiple, overlapping rules — attendance percentage, internal marks, and condonation categories — is slow, repetitive, and prone to human error. This project automates that evaluation process using core programming concepts: conditionals, loops, functions, lists, dictionaries, sorting, and searching.

Scope

The project covers storing and validating student records, evaluating eligibility using tiered rule-based logic, generating individual and class-wide summary reports, and providing sorting and searching utilities. It does not include a graphical interface, a persistent database, or web access — it is a menu driven command-line application, which matches the level of an introductory programming course.

Target Users

Faculty members and academic administration staff who need a quick, automated way to determine which students are eligible, need condonation, or are debarred from examinations

Functional Requirements

The system is organised into three major functional modules, as required by the project brief:

Module 1 — Student Data Management

- Store student records (roll number, name, attendance %, internal marks) as a list of dictionaries.
- Validate new records before adding them — attendance must be between 0 and 100, internal marks within a valid range.
- Display all currently stored student records.

Module 2 — Eligibility Evaluation Engine

- Check attendance against three tiers: Eligible ($\geq 75\%$), Condonation Required (65–74.9%), Debarred ($< 65\%$).
- Check internal marks against a minimum threshold
- Combine both checks into one final status per student, e.g. "Debarred - Attendance", "Condonation Required"

Module 3 — Report Generation & Analytics

- Print an individual report for every student
- Print a class-wide summary (counts of Eligible / Condonation / Debarred)
- Filter and sort debarred students by roll number using a custom bubble sort
- Search for a specific student by roll number using linear search
- Export the full report to a text file.

Together with main.py (the menu that ties the three modules together), the project has 7 Python files: main.py, student_data.py, eligibility_engine.py, report_generator.py, sorting_utils.py, search_utils.py, and test_eligibility.py.

Non-Functional Requirements

Requirements	How it's addressed
Usability	Simple numbered menu (options 1–7); clear, formatted console output

Scalability	Threshold values are kept as named constants at the top of <code>eligibility_engine.py</code> , so rules can be changed without touching the logic
Reliability	Invalid menu choices and non-numeric input are handled without crashing the program
Maintainability	Code is split into small, single-purpose modules and functions with descriptive names
Error Handling	try/except around numeric input in the "Add student" flow; validation before storing any record
Performance	All operations are simple loops over an in-memory list, which is efficient for the class sizes this tool targets

System Architecture

The system follows a simple three-layer architecture: a data layer, a processing/evaluation layer, and a reporting layer, all coordinated by a single entry point (`main.py`).

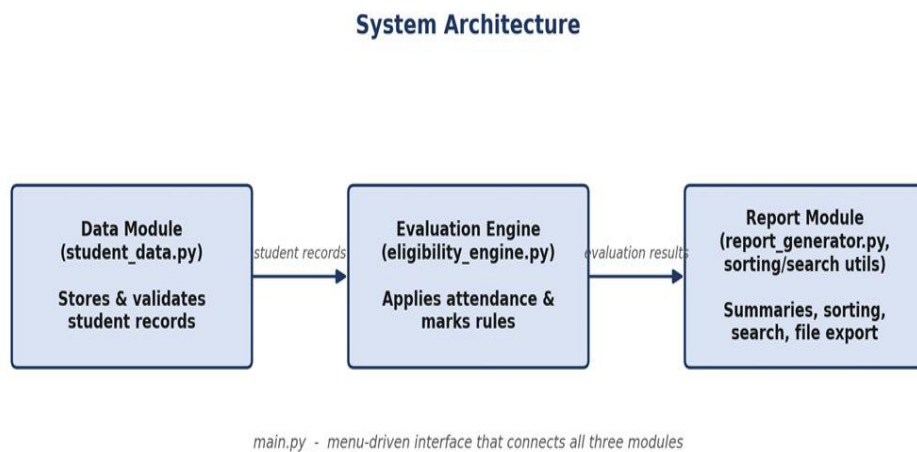


Figure 1: System Architecture — data flows from the Data Module through the Evaluation Engine to the Report Module.

Design Diagrams

Use Case Diagram

The only actor in this system is the Faculty/Admin user, who interacts with the program through the menu.

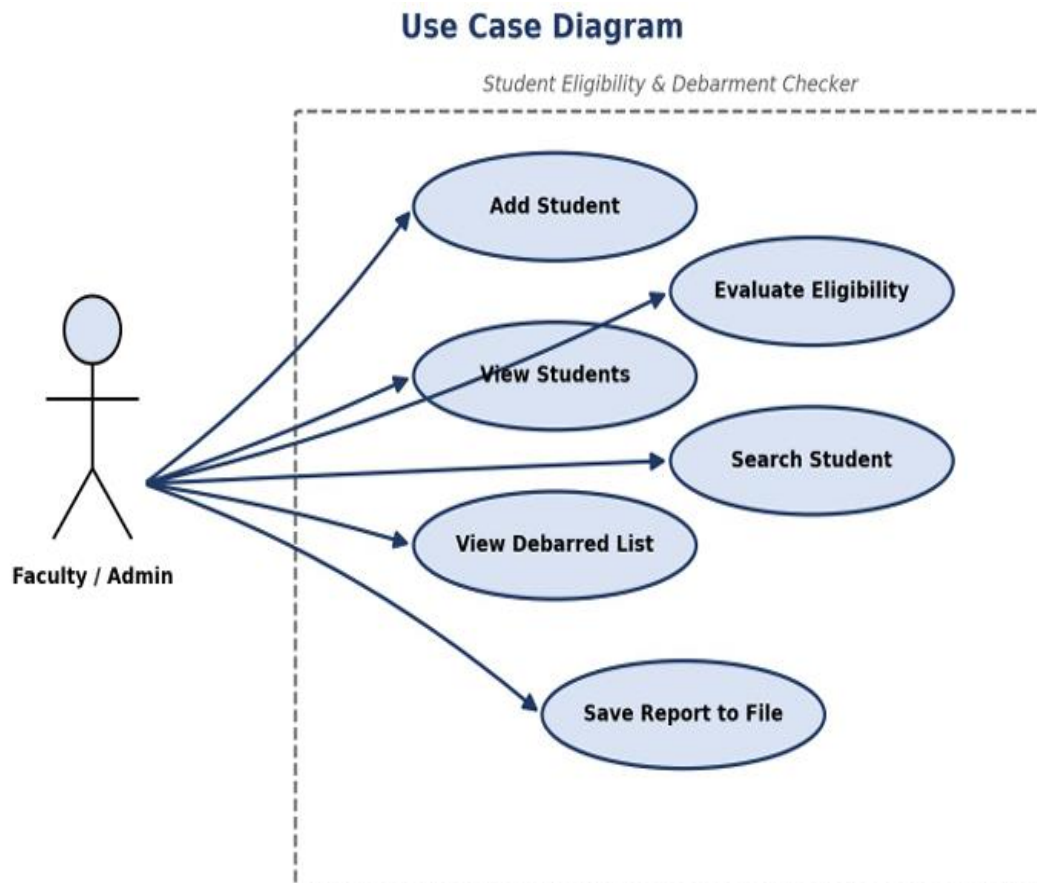


Figure 2: Use Case Diagram

Process Flow / Workflow Diagram

This flowchart traces what happens from the moment the program starts to when the user exits.

Process Flow / Workflow

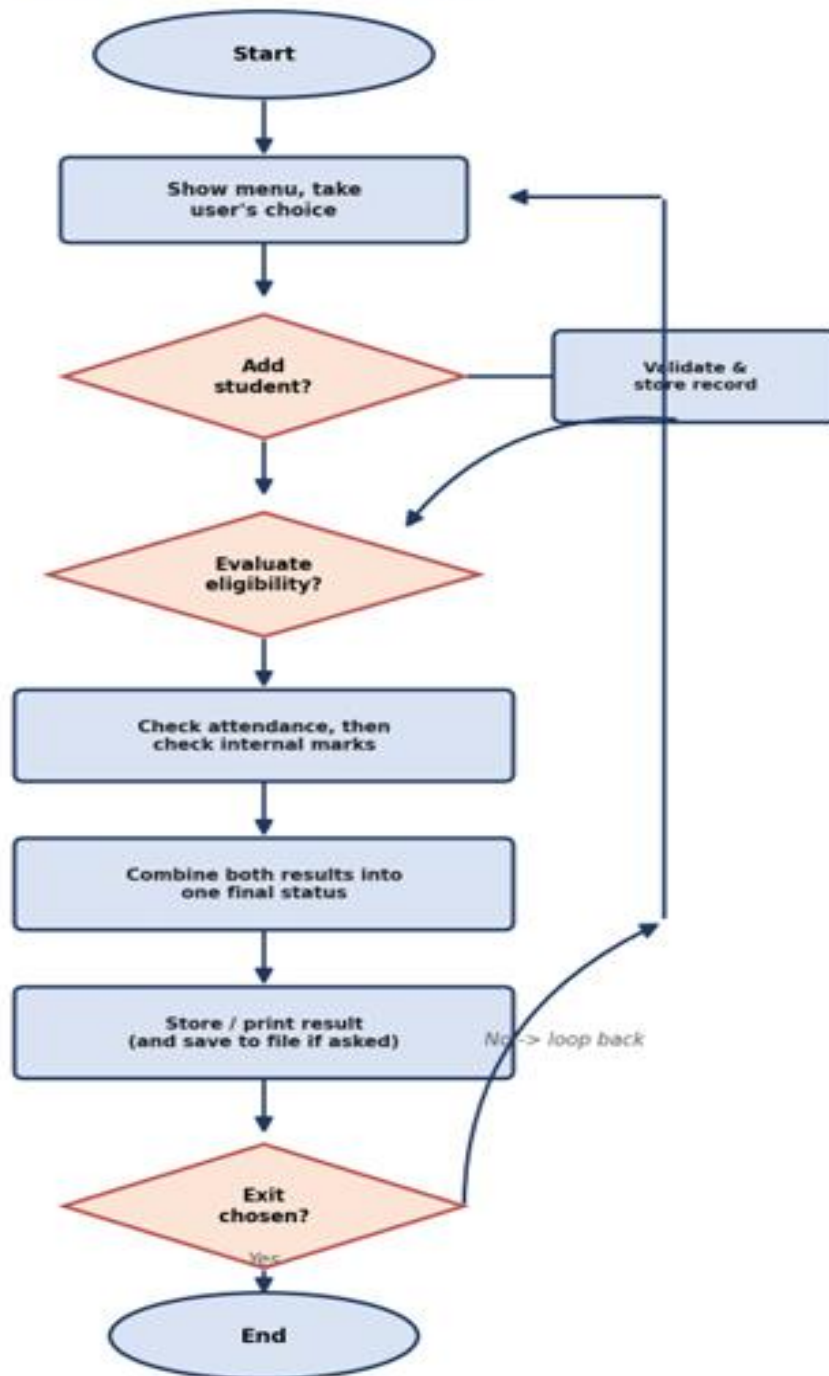


Figure 3: Process Flow / Workflow Diagram

Component / Module Diagram

Since this project does not use classes, a component diagram is shown instead, illustrating how the modules depend on one another through imports.

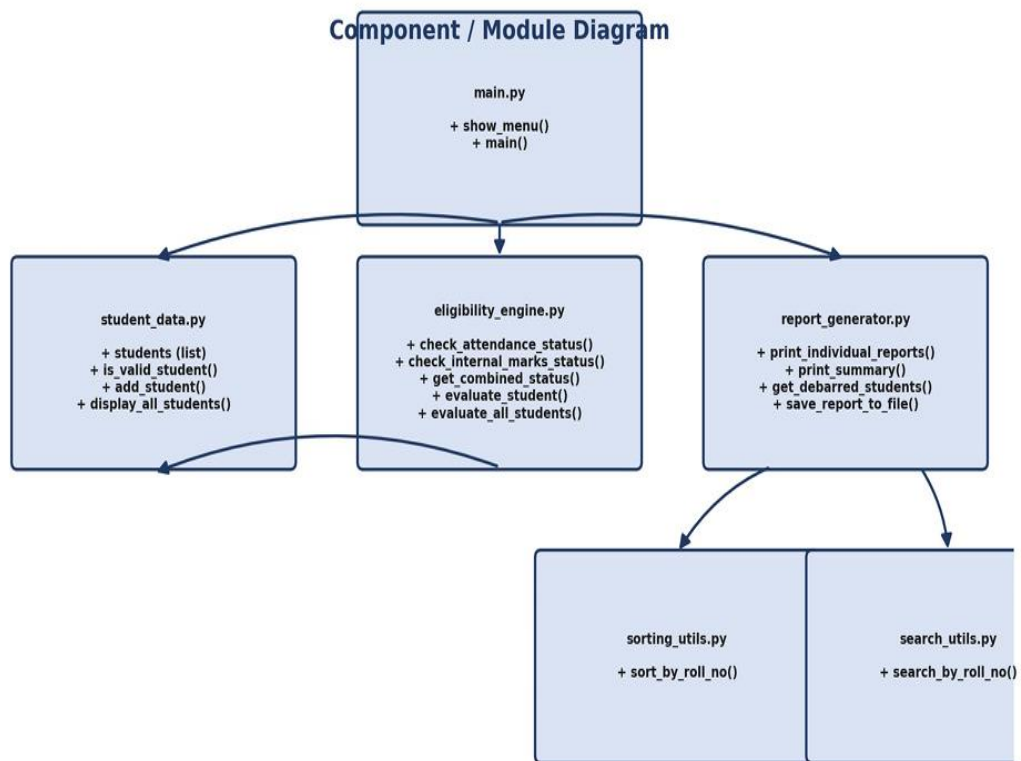


Figure 4: Component Diagram — arrows show which module imports from which.

Sequence Diagram

This sequence diagram shows the flow of messages between modules when the user selects option 3 (run eligibility evaluation) from the menu.

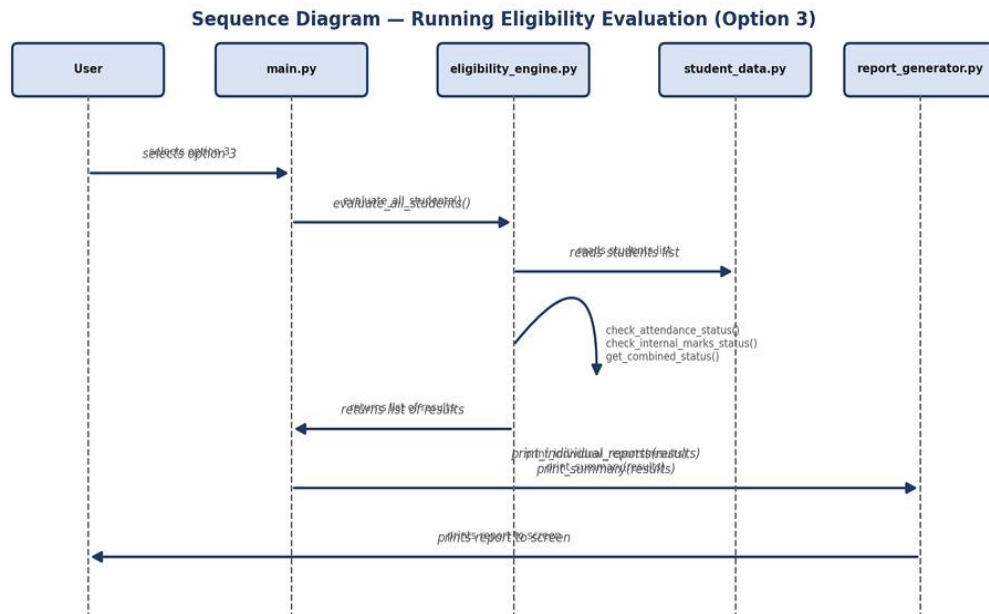


Figure 5: Sequence Diagram — Running Eligibility Evaluation

Database / ER Diagram

This project does not use a database or any persistent storage — all student records are held in memory as a list of dictionaries for the duration of the program. An ER diagram is therefore not applicable here; this is a deliberate scope decision explained further in the Design Decisions section below.

Design Decisions & Rationale

- Dictionaries over parallel lists: each student is stored as a single dictionary (roll_no, name, attendance, internal_marks) rather than four separate lists. This keeps each student's data together and makes the code far more readable, e.g. `student["attendance"]` instead of tracking matching indexes across lists.
- Custom bubble sort instead of Python's built-in `sort()`: Unit 5 of the syllabus specifically covers array/list sorting techniques, so `sort_by_roll_no()` is implemented manually to demonstrate the underlying algorithm rather than hiding it behind a built-in function.
- Linear search instead of a built-in lookup: similarly, `search_by_roll_no()` loops through the results one at a time, since this is the searching technique the course covers.

- Threshold values as named constants: ATTENDANCE_ELIGIBLE, ATTENDANCE_CONDONATION, and MIN_INTERNAL_MARKS are defined once at the top of eligibility_engine.py, so the rules can be adjusted in one place if the college's actual policy differs.
- No database or GUI: the course does not cover either, so keeping the project as a command-line, in memory application keeps the implementation honestly within the concepts taught, rather than adding complexity for its own sake.
- Splitting sorting and searching into their own files (sorting_utils.py, search_utils.py): this keeps report_generator.py focused on formatting and printing, and makes each algorithm easy to find and test on its own.

Implementation Details

student_data.py

Holds the students list (list of dictionaries) and two supporting functions: is_valid_student(), which checks that attendance is within 0–100 and internal marks are within a valid range, and add_student(), which builds a new record and only appends it to the list if it passes validation. display_all_students() prints every record in a readable format.

eligibility_engine.py

This is where the core decision-making logic lives. check_attendance_status() applies the three-tier attendance rule using if–elif–else; check_internal_marks_status() applies the minimum-marks rule; and get_combined_status() merges both results into a single final status. evaluate_student() runs both checks for one student, and evaluate_all_students() loops over the whole list and returns the results.

sorting_utils.py and search_utils.py

sort_by_roll_no() implements bubble sort: it repeatedly steps through the list, compares adjacent roll numbers, and swaps them if they're in the wrong order, until the whole list is sorted. search_by_roll_no() implements linear search: it checks each record one at a time until it finds a match, or reaches the end of the list.

report_generator.py

Uses the evaluation results to print an individual report per student, count how many students fall into each status category (print_summary()), filter out just the debarred students (get_debarred_students()), and write the full report out to eligibility_report.txt (save_report_to_file()).

main.py

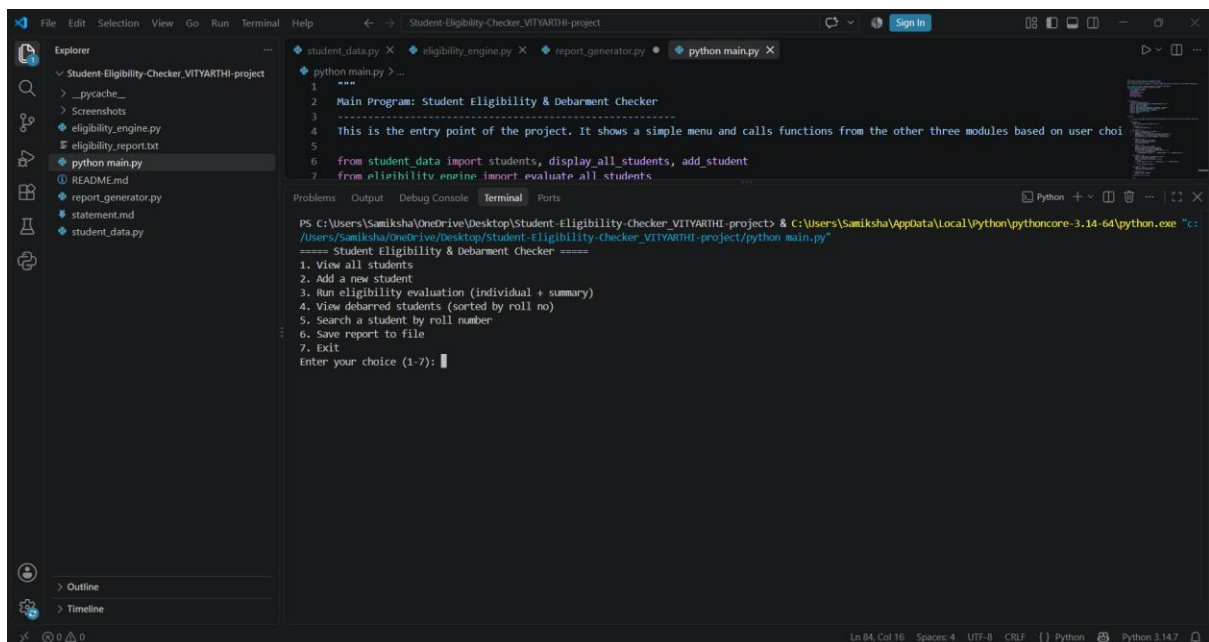
Ties everything together behind a simple numbered menu inside a while True loop. Option 2 (add a new student) is wrapped in a try/except block, so that if the user accidentally types letters instead of numbers for attendance or marks, the program prints a clear error message instead of crashing. Choosing option 7 breaks out of the loop and ends the program cleanly.

test_eligibility.py

A standalone test file (see Testing Approach below) that checks the eligibility rules, validation, sorting, and searching all behave correctly, including at the boundary values of each threshold.

Screenshots / Results

The following screenshots show the program running end to end.



```
1  ***
2  Main Program: Student Eligibility & Debarment Checker
3  -----
4  This is the entry point of the project. It shows a simple menu and calls functions from the other three modules based on user choi
5
6  from student_data import students, display_all_students, add_student
7  from eligibility_engine import evaluate_all_students
```

```
PS C:\Users\Samiksha\OneDrive\Desktop\Student-Eligibility-Checker_VITYARTHI-project> C:\Users\Samiksha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/Samiksha/OneDrive/Desktop/Student-Eligibility-Checker_VITYARTHI-project/python_main.py"
===== Student Eligibility & Debarment Checker =====
1. View all students
2. Add a new student
3. Run eligibility evaluation (individual + summary)
4. View debarred students (sorted by roll no)
5. Search a student by roll number
6. Save report to file
7. Exit
Enter your choice (1-7): 1
```

Figure 6: student_data.py — sample records displayed, then a new student added

```

1  """
2  Main Program: Student Eligibility & Debarment Checker
3  -----
4  This is the entry point of the project. It shows a simple menu and calls functions from the other three modules based on user choi
5
6  from student_data import students, display_all_students, add_student
7  from eligibility_engine import evaluate_all_students

```

```

1. View all students
2. Add a new student
3. Run eligibility evaluation (individual + summary)
4. View debarred students (sorted by roll no)
5. Search a student by roll number
6. Save report to file
7. Exit
Enter your choice (1-7): 3
--- Individual Student Report ---
210H1001 | Barry | Eligible
210H1002 | Mishah | Debarred - Attendance
210H1003 | Anushka | Condonation Required
210H1004 | Samanyu | Eligible
210H1005 | Rishiika | Debarred - Attendance
--- Class Summary ---
Total Students: 5
Eligible: 2
Condonation Required: 1
Debarred: 2
===== Student Eligibility & Debarment Checker =====
1. View all students
2. Add a new student
3. Run eligibility evaluation (individual + summary)
4. View debarred students (sorted by roll no)
5. Search a student by roll number
6. Save report to file
7. Exit
Enter your choice (1-7):

```

Figure 7: eligibility_engine.py — attendance and marks evaluated for each sample student

```

1  """
2  Validation Tests
3  -----
4  Simple tests to check that the eligibility rules, validation,
5  sorting and searching behave correctly, including boundary values.
6
7  Run with: mvthon test_eligibility.py

```

```

PS C:\Users\Samiksha\OneDrive\Desktop\Student-Eligibility-Checker_VITYARTHI-project> C:\Users\Samiksha\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/Samiksha/OneDrive/Desktop/Student-Eligibility-Checker_VITYARTHI-project/test_eligibility.py
PASS: Attendance 90 -> Eligible
PASS: Attendance 75 (boundary) -> Eligible
PASS: Attendance 74.9 -> Condonation
PASS: Attendance 65 (boundary) -> Condonation
PASS: Attendance 64.9 -> Debarred
PASS: Attendance 0 -> Debarred
PASS: Marks 45 -> Eligible
PASS: Marks 20 (boundary) -> Eligible
PASS: Marks 19 -> Debarred
PASS: Both fine -> Eligible
PASS: Condonation + good marks -> Condonation Required
PASS: Low attendance only -> Debarred - Attendance
PASS: Low marks only -> Debarred - Internal Marks
PASS: Both low -> Debarred - Attendance & Internals
PASS: Valid student accepted
PASS: Attendance above 100 rejected
PASS: Negative marks rejected
PASS: Bubble sort orders roll numbers
PASS: Search finds existing roll number
PASS: Search returns None for missing roll number

Result: 20 passed, 0 failed
PS C:\Users\Samiksha\OneDrive\Desktop\Student-Eligibility-Checker_VITYARTHI-project>

```

Figure 8: report_generator.py — individual report, class summary, sorted debarred list, search result, and file export

Testing Approach

Testing was done in two ways: manual testing through the menu (shown in the screenshots above) and a dedicated automated test file, `test_eligibility.py`, which checks the core logic directly without needing to go through the menu each time.

The automated tests specifically check boundary values, since these are the places where off-by-one mistakes in `if/elif` conditions are most likely

Test case	Attendance / Marks	Expected result
Just above eligible threshold	75%	Eligible
Just below eligible threshold	74.9%	Condonation Required
Just above condonation threshold	65%	Condonation Required
Just below condonation threshold	64.9%	Debarred
Minimum internal marks (boundary)	20 marks	Eligible
One mark below minimum	19 marks	Debarred

In total, the test file runs 20 checks covering the attendance rule, the internal marks rule, the combined status logic, input validation, the custom sort, and the custom search. All 20 pass. The test file also confirms that invalid data (e.g. attendance above 100, or negative marks) is correctly rejected by `is_valid_student()`.

Manual testing through the menu also confirmed that typing letters instead of numbers when adding a student (a common user mistake) is caught by the `try/except` block and shown as a clear error message, rather than crashing the program.

Challenges Faced

Coming into this project, I was still fairly new to Python — comfortable with variables and print statements, but not yet confident with loops and conditionals, and I had never used Git, GitHub, or an IDE like VS Code before. A few specific things I ran into:

- Setting up the environment: I initially tried running Python through IDLE and PowerShell before settling on VS Code, and it took a bit of trial and error to understand the difference between just opening a Python interpreter versus actually creating and running a `.py` file inside a project folder.
- Understanding imports across files: splitting the project into multiple files (`main.py`, `student_data.py`, etc.) meant learning how from module import function actually works, and why all the files need to sit in the same folder for that to succeed.
- Indentation errors: Python is strict about indentation in a way I wasn't used to. When I rewrote some of the comments in my own words by copy-pasting code out of

the editor and back in, the indentation was lost, which broke the file until I went back through it function by function.

- Markdown image paths in README.md: my screenshots were saved in different folders than I expected, so the image links in my README didn't resolve at first. I had to move everything into one consistent screenshots folder and match the file names exactly to the paths written in the Markdown.
- Git and GitHub from scratch: I'd never used version control before, so learning the basic sequence of git init, git add, git commit, and git push — and getting my GitHub account set up under a personal email instead of my school one — was new territory.

Learnings & Key Takeaways

- Breaking a problem into small functions (one job per function) made the logic much easier to test and debug than one long block of code would have been.
- Conditional logic order matters — when using if/elif/else, the first condition that matches wins, so rules have to be ordered carefully (e.g. checking the highest attendance tier first).
- Writing tests for boundary values (like exactly 75% or exactly 65%) caught exactly the kind of off by-one mistake that's easy to make and easy to miss just by eyeballing the code.
- Splitting a program across multiple files makes it more organised, but only if you understand how imports connect those files — otherwise it's easy to end up with a program that looks complete but doesn't actually run.
- Version control (Git/GitHub) is worth learning early — even for a small project, having a commit history makes it possible to see how the project evolved, not just the final state.

Future Enhancements

- Persistent storage: save student records to a file or a database so data isn't lost when the program closes.
- Graphical interface: a simple Tkinter GUI instead of a text menu, to make the tool more approachable for non-technical staff.
- Configurable thresholds: read the attendance/marks cutoffs from a config file instead of hardcoded constants, so different departments could apply different rules.

- Duplicate roll number checks: currently the same roll number could be added twice; this could be validated against in `add_student()`.
- Email/notification integration: automatically notify debarred students or their advisors.

References

- Dromey, R. G. — How to Solve it by Computer, PHI.
- Downey, Allen B. — Think Python: How to Think Like a Computer Scientist (2nd ed.), O'Reilly, 2015.
- Kanetkar, Yashavant — Let Us Python (1st ed.), BPB Publishers, 2019.
- Python 3 official documentation — docs.python.org
- CSE1021 – Introduction to Problem Solving and Programming, course syllabus.